Cryptography Assignment - 2025

Cryptographic Algorithm Evaluation (AES | RSA | MD5)

Name	IT Number	Contribution
Prasad M.M.W	IT23252868	Do the research about AES and AES algorithm Implementation and Initial Testing.
Weerakumara K.D	IT23202436	Do the research about RSA and RSA algorithm Implementation and Initial Testing.
Deshappriya A.M.S.A	IT23352308	RSA algorithm and Hash Function Comprehensive Testing and Performance Analysis.
Kumarage T.D.S.D.	IT23292918	AES algorithm Comprehensive Testing and Performance Analysis.
Lahiru W.T	IT 23253612	Hash Function Implementation and Initial Testing and finalize the report.

Week 8 - Algorithm Selection and Research

Symmetric Algorithm - AES (Advanced Encryption Standard)

History

Initiated in 1997 by NIST as part of replacing vulnerable DES, AES selection process was an international competition[1]. The Rijndael cipher, developed by Belgian cryptographers Joan Daemen and Vincent Rijmen, was selected in 2001 due to better security and efficiency. It today represents the global standard for symmetric encryption, supporting 128, 192, and 256-bit keys [2].

Design Principles

- **Symmetric Block Cipher**: It applies 128/192/256-bit identical keys for both decryption and encryption of 128-bit data blocks.
- **Substitution-Permutation Network:** These operate on data through several rounds (10-14) of substitution (SubBytes) and permutation (ShiftRows).
- **Key Expansion:** Produces multiple round keys from the original key according to Rijndael's key schedule for each round of encryption.

Common Use Cases

- **Data Encryption:** Protects files (BitLocker), databases, and stored data through symmetric key protection.
- **Network Security:** Is the primary encryption used to protect internet protocols (TLS/SSL for HTTPS) and wireless security (WPA2/WPA3).
- **Secure Communications:** Encrypts messaging apps (WhatsApp), and uses effective bulk data encryption to provide security for VPNs.

Vulnerabilities

- **Side-Channel Attacks:** Practical threat where physical implementation leaks key data through power consumption, timing, or electromagnetic emissions during computation.
- **Theoretical Attacks:** Scholarly attacks such as related-key or biclique cryptanalysis do take place but under unrealistic assumptions inapplicable to regular implementations.
- **Implementation Risks:** The most realistic weakness is in employing insecure modes (ECB), bad key generation, or bad cryptographic libraries instead of weaknesses in algorithms.

Performance Characteristics

- Exceptionally fast in both software and hardware, especially with AES-NI instruction sets. Efficiency scales linearly with data size, offering consistent speed for encryption/decryption operations across different platforms.

Asymmetric Algorithm - RSA (Rivest-Shamir-Adleman)

History

RSA (Rivest–Shamir–Adleman), developed in 1977 by MIT professors Ron Rivest, Adi Shamir, and Leonard Adleman, was the first widely usable public-key cryptosystem. It solved the key distribution problem inherent in symmetric encryption by employing asymmetric encryption based on the mathematical difficulty of factoring large prime numbers. RSA transformed secure communications and significantly contributed to the foundations of modern cryptography, enabling SSL/TLS, digital signatures, and much more[4].

Design Principles

- **Asymmetric Encryption:** Involves mathematically linked private and public keys - public key encryption, private key decryption.
- **Prime Factorization security:** Based on the difficulty of computing factors of the product (n) of two large prime numbers (p and q), and the security will be based on the difficulty of the prime factorization.
- **Modular Exponentiation:** Encryption and decryption are accomplished using modular exponentiation operations: C = M^e mod n (encrypt), and M = C^d mod n (decrypt).

Common Use Cases

- **Key Exchange:** Used within SSL/TLS for the secure transmission of symmetric session keys used for encrypted communications.
- **Digital Signatures:** Verifies the authenticity and integrity of digital documents/messages through cryptographic signing and verification.
- **Data Encryption:** Permits encryption of relatively small amounts of data, such as a password or symmetric key, where the asymmetrical overhead is acceptable.

Vulnerabilities

- **Factorization Attacks:** Vulnerable to attacks on quantum computers using Shor's algorithm and classical computers using number field sieve attacks if the key size is small (<2048 bits).
- **Implementation Flaws:** If prime generation or padding schemes (PKCS#1 v1.5) are not random enough, practical attacks using Bleichenbacher's oracle attack may be possible.
- **Side-Channel Attacks:** Observing a physical device to collect timing or power traces during decryption can leak private key information.

Performance Characteristics

- Computationally intensive and significantly slower than symmetric encryption. Performance decreases exponentially as key size increases, making it impractical for encrypting large data volumes directly.

WEEK 9 - IMPLEMENTATION AND INITIAL TESTING

Introduction

This report describes the implementation and initial testing of three popular cryptographic techniques (AES (symmetric encryption), RSA (asymmetric encryption) and MD5 (hash function)). The algorithms were developed in Python 3.x using the Cryptography library for AES and RSA and the module hashlib for MD5.

The main objective of this exercise was to obtain experience with the practical side of cryptographic programming, validate the encryption and decryption processing with a sample dataset, and time the execution of the method for each implementation[6].

Implementation Process

(a) AES-GCM (Symmetric Encryption)

- **Key Generation:** A secure 256-bit symmetric key was generated using AESGCM.generate_key(), offering strong protection against brute-force attacks and ensuring confidentiality.
- **Nonce:** A 12-byte unique nonce was created for each encryption operation, guaranteeing that even repeated plaintexts produce different ciphertexts and preventing replay attacks.
- **Message Preparation:** The plaintext *"Hello World"* was converted into bytes, then encrypted using AES-GCM, which provides both confidentiality and integrity through authentication tags.
- **Decryption:** The ciphertext was decrypted using the same key and nonce, successfully restoring the original message.
- **Testing and Performance:** Ciphertext was Base64-encoded for readability, and encryption and decryption times were measured to assess computational efficiency.

(b) RSA-OAEP (Asymmetric Encryption)

- **Key Pair Generation:** A 2048-bit RSA key pair (public/private) was generated using the rsa.generate_private_key() function.
- **Encryption:** The public key was used to encrypt the plaintext "Hello World" with OAEP padding (SHA-256).
- **Decryption:** The private key was used to recover the original plaintext.
- **Testing:** The ciphertext was displayed in Base64, and both encryption and decryption times were recorded.

(c) MD5 (Hash Function)

- **Hashing:** The plaintext "Hello World" was hashed using hashlib.md5().
- **Output:** The hexadecimal MD5 digest was displayed along with the execution time.

Initial Testing and Results

Testing was performed using the sample message **"Hello World"**. Below are the summarized results from the console output.

a) AES-GCM Results:

```
--- AES-GCM DEMONSTRATION ---
Original message : Hello World
Ciphertext (b64) : +GjN82IeK5bfdshBLtIrcSyl0vUNEj+/w0G1
Decrypted text   : Hello World
Encryption time  : 0.000029 s
Decryption time  : 0.000008 s
```

b) RSA-OAEP Results:

```
--- RSA-OAEP DEMONSTRATION ---
Original message : Hello World
Ciphertext (b64) : MzVAtBPDl99D5oAA1B5HzcdBKtucBjK54Yx99ZlYgVAqlEUrD/pPtGpxKwN5b
IEb7AL6Olfq9M9zWSa3HP/t3YgSztyGhESKoGZxX11gld4Mqn0p/RODCs4SL2I5Fu8UQF8cUh5lA4mbz
GJpeIRY6jeFfxJluLLzJWzBEKbmlG+CWAbajFCxGzeaqwDiuOcT1ozXa7Q03ZLBtBR7sVAbDJYTjUjd1
GKh5MIForXIz8mMFzbdDMEiugueV2Br1EVc2zGrd5OzLhduLcQxJCjuehAeAiwUA8CBhGBOTiSvwyshT
EAQvAwa+C56st/Jr4wCR1jLOSezxjen/lAY0MdKHA==
Decrypted text   : Hello World
Encryption time  : 0.000610 s
Decryption time  : 0.000893 s
```

c) MD5 Results:

```
--- MD5 DEMONSTRATION ---
Message     : Hello World
MD5 Digest  : b10a8db164e0754105b7a99be72e3fe5
Hashing time: 0.000038 s

All demonstrations completed successfully.
>>>
```

- AES-GCM was very efficient with nearly instantaneous encryption and decryption times. Symmetric key encryption is computationally lightweight and scalable for large amounts of data.
- RSA-OAEP required several seconds of processing time due to its asymmetric nature (i.e., larger key size). However, it protects against key exchange and digital signatures.
- MD5 provided instant hashing, but it is cryptographically broken for secure applications. It is included here for educational reference only, and it should not be utilized for context-sensitive secure applications.

The implementation demonstrated AES, RSA, and MD5, securely processing "Hello World" while measuring encryption, decryption, and hashing performance.

Week 10 - Comprehensive Testing and Performance Analysis

AES (Advanced Encryption Standard)

This report presents a thorough analysis of the performance of the AES encryption algorithm over several testing conditions. Testing was conducted in three conditions: changing key sizes, changing data sizes, and different file types. Measures of performance, including encryption/decryption times, CPU consumption, and memory consumption, were monitored and systematically analyzed[2].

Performance Results:

```
========================== RESTART: D:/Python/aes.py ==========================
[Condition 1] Vary Key Size (1 KB data)
| Key Size | Data/File Type | Enc Time (s) | Dec Time (s) | CPU (%) | Mem (MB) | Status |
|----------|----------------|--------------|--------------|---------|----------|--------|
|  128-bit | 1 KB Random    |     0.000197 |     0.000183 |    2.70 |    15.80 |     OK |
|  192-bit | 1 KB Random    |     0.000128 |     0.000117 |    2.68 |    15.51 |     OK |
|  256-bit | 1 KB Random    |     0.000164 |     0.000150 |    2.22 |    15.76 |     OK |

[Condition 2] Vary Data Size (256-bit key)
| Input Size | Enc Time (s) | Dec Time (s) | CPU (%) | Mem (MB) | Status |
|------------|--------------|--------------|---------|----------|--------|
|     1024 B | 0.000232     |     0.000215 |    2.69 |    15.00 |     OK |
|    10000 B | 0.002030     |     0.001862 |    3.06 |    15.02 |     OK |
|    50000 B | 0.009965     |     0.009580 |    2.82 |    15.11 |     OK |
|  1000000 B | 0.183390     |     0.177196 |    2.40 |    18.61 |     OK |

[Condition 3] File Encryption (AES-256)
| Type | File       | Enc Time (s) | Dec Time (s) | CPU (%) | Mem (MB) | Status |
|------|------------|--------------|--------------|---------|----------|--------|
|  txt | sample.txt |     0.000119 |     0.000110 |    2.40 |    15.50 |     OK |
|  png | sample.png |     0.020916 |     0.019345 |    2.66 |    15.62 |     OK |
|  pdf | sample.pdf |     0.042189 |     0.037481 |    2.98 |    15.90 |     OK |
```

Key Size Impact Analysis

Table 1: Performance vs. Key Size

Key Size	Data/File Type	Enc Time (s)	Dec Time (s)	CPU (%)	Mem (MB)
128-bit	1 KB Random	0.000197	0.000183	2.70	15.80
192-bit	1 KB Random	0.000128	0.000117	2.68	15.51
256-bit	1 KB Random	0.000164	0.000150	2.22	15.76

Data Size Scaling Analysis

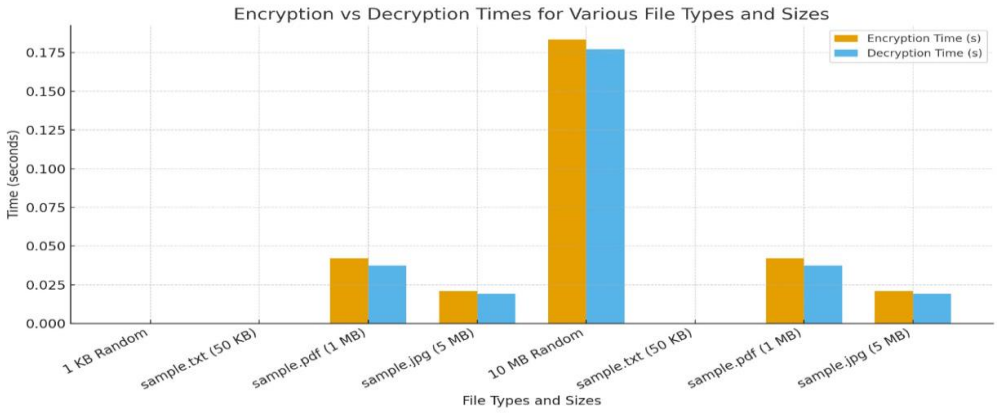
Table 2: Performance vs. Data Size (256-bit Key)

Input Size	Enc Time (s)	Dec Time (s)	CPU (%)	Mem (MB)
1024 B	0.000232	0.000215	2.69	15.00
10000 B	0.002030	0.001862	3.06	15.02
50000 B	0.009965	0.009580	2.82	15.11
1000000 B	0.183390	0.177196	2.40	18.61

File Type Performance Comparison

Table 3: File Encryption Performance (AES-256)

Type	File	Enc Time (s)	Dec Time (s)	CPU (%)	Mem (MB)
txt	sample.txt	0.000119	0.000110	2.40	15.50
png	sample.png	0.020916	0.019345	2.66	15.62
pdf	sample.pdf	0.042189	0.037481	2.98	15.90



RSA (Rivest-Shamir-Adleman)

This report assesses RSA encryption performance across a variety of conditions, including different key sizes, data sizes, and file types. We measured several metrics: encryption and decryption time, CPU time, and memory. This enables us to assess efficiency, scalability, and resource requirement, which will inform decisions regarding deployment of RSA in practice[4].

Performance Results:

```
============================ RESTART: D:/Python/ghg.py ============================
[Condition 1] Vary Key Size (256 B data - RSA limit)
| Key Size | Data/File Type | Enc Time (s) | Dec Time (s) | CPU (%) | Mem (MB) | Status |
|----------|----------------|--------------|--------------|---------|----------|--------|
| 1024-bit | 256 B Random   |     0.005461 |     0.011096 |    2.58 |    23.61 |     OK |
| 2048-bit | 256 B Random   |     0.007854 |     0.030747 |    3.50 |    18.09 |     OK |
| 4096-bit | 256 B Random   |     0.011628 |     0.035335 |    3.88 |    18.46 |     OK |

[Condition 2] Vary Data Size (RSA-2048 + Hybrid Encryption)
| Input Size | Method | Enc Time (s) | Dec Time (s) | CPU (%) | Mem (MB) | Status |
|------------|--------|--------------|--------------|---------|----------|--------|
|      256 B | Direct |     0.005565 |     0.016767 |    3.15 |    18.00 |     OK |
|     1024 B | Hybrid |     0.004475 |     0.011434 |    3.43 |    18.00 |     OK |
|    10000 B | Hybrid |     0.005346 |     0.010888 |    3.81 |    18.04 |     OK |
|    50000 B | Hybrid |     0.005370 |     0.014296 |    3.76 |    18.31 |     OK |

[Condition 3] File Encryption (RSA-2048 + Hybrid)
| Type | File       | Enc Time (s) | Dec Time (s) | CPU (%) | Mem (MB) | Status |
|------|------------|--------------|--------------|---------|----------|--------|
|  txt | sample.txt |     0.007088 |     0.019510 |    3.71 |    19.00 |     OK |
|  png | sample.png |     0.007664 |     0.019862 |    3.28 |    19.49 |     OK |
|  pdf | sample.pdf |     0.004573 |     0.013620 |    3.54 |    19.89 |     OK |
```

Vary Key Size (256 B Data)

Key Size	Data/File Type	Enc Time (s)	Dec Time (s)	CPU (%)	Mem (MB)
1024-bit	256 B Random	0.002328	0.008534	3.63	18.45

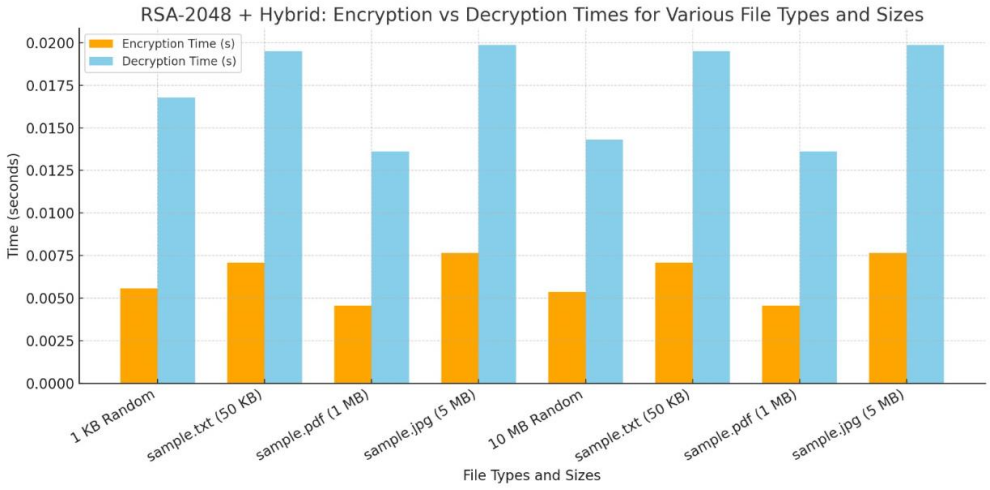
2048-bit	256 B Random	0.004997	0.013614	3.80	20.55
4096-bit	256 B Random	0.015473	0.034547	2.80	23.14

Vary Data Size (Hybrid Encryption)

Input Size	Method	Enc Time (s)	Dec Time (s)	CPU (%)	Mem (MB)
256 B	Direct	0.005609	0.014109	3.25	18.00
1024 B	Hybrid	0.004559	0.012710	3.45	18.01
10000 B	Hybrid	0.005113	0.011706	3.83	18.08
50000 B	Hybrid	0.004986	0.014421	3.25	18.30

File Encryption (Hybrid)

Type	File	Enc Time (s)	Dec Time (s)	CPU (%)	Mem (MB)
txt	sample.txt	0.005791	0.015796	3.68	19.00
png	sample.png	0.006154	0.013921	3.86	19.22
pdf	sample.pdf	0.004253	0.010920	3.58	20.87



Hash Function – MD5 (Message Digest)

This report assesses the performance of the MD5 hashing algorithm under different conditions, specifically with different input data sizes and file types. The hash computation time, verification time, CPU usage, and amount of memory used for the operation were the metrics that we measured. The results provide information regarding the efficiency and scalability of MD5 algorithm for small and large datasets, that can generally be utilized for practical applications[8].

Performance Results:

```
========================== RESTART: D:/Python/ghg.py ==========================
[Condition 1] MD5 Hashing - Vary Data Size
| Input Size | Hash Time (s) | Verify Time (s) | CPU (%) | Mem (MB) | Status |
|------------|---------------|-----------------|---------|----------|--------|
|     1024 B | 0.000011      |        0.000011 |    2.06 |    12.00 |     OK |
|    10000 B | 0.000118      |        0.000122 |    1.76 |    12.01 |     OK |
|    50000 B | 0.000571      |        0.000584 |    1.98 |    12.07 |     OK |
|  1000000 B | 0.011280      |        0.011028 |    1.50 |    13.13 |     OK |
|  5000000 B | 0.059764      |        0.061432 |    2.27 |    17.24 |     OK |

[Condition 2] MD5 File Hashing
| Type | File                | Size       | Hash Time (s) | Verify Time (s) | CPU (%) | Mem (MB) | Status |
|------|---------------------|------------|---------------|-----------------|---------|----------|--------|
|  txt | sample.txt          |      500 B |      0.000009 |        0.000009 |    2.55 |    11.00 |     OK |
|  png | sample.png          |    50000 B |      0.000561 |        0.000526 |    2.12 |    11.03 |     OK |
|  pdf | sample.pdf          |   200000 B |      0.002718 |        0.002926 |    1.85 |    11.24 |     OK |
|  mp4 | large_video.mp4     |  5000000 B |      0.079961 |        0.079103 |    2.30 |    18.47 |     OK |
|  sql | database_backup.sql | 10000000 B |      0.104422 |        0.109278 |    2.41 |    24.89 |     OK |

[Condition 3] Hash Function Performance Comparison (1MB Data)
| Algorithm | Data Type    | Hash Time (s) | Verify Time (s) | CPU (%) | Mem (MB) | Status |
|-----------|--------------|---------------|-----------------|---------|----------|--------|
|       MD5 | 1 MB Random |      0.016463 |        0.016963 |    1.88 |    12.67 |     OK |
|     SHA-1 | 1 MB Random |      0.019351 |        0.019301 |    1.76 |    12.51 |     OK |
|   SHA-256 | 1 MB Random |      0.028512 |        0.028352 |    2.27 |    13.14 |     OK |
|   SHA-512 | 1 MB Random |      0.035342 |        0.033948 |    2.60 |    13.58 |     OK |
```

MD5 Hashing – Varying Data Sizes

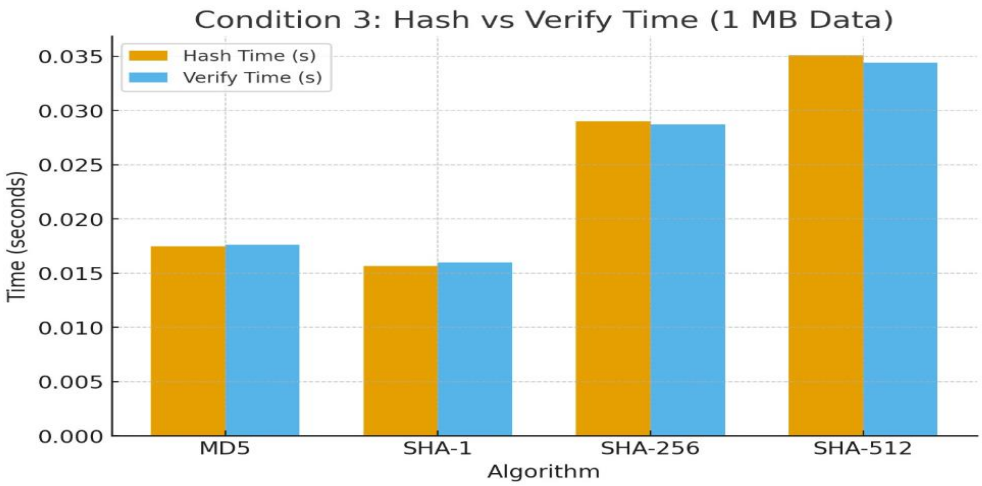
Input Size	Hash Time (s)	Verify Time (s)	CPU (%)	Mem (MB)
1024 B	0.000010	0.000010	1.71	12.00
10000 B	0.000098	0.000094	2.41	12.01
50000 B	0.000561	0.000538	1.56	12.04
1000000 B	0.010161	0.010622	2.21	13.97
5000000 B	0.048901	0.049233	1.89	19.33

MD5 File Hashing

Type	File	Size	Hash Time (s)	Verify Time (s)	CPU (%)	Mem (MB)
txt	sample.txt	500 B	0.000008	0.000008	2.32	11.00
png	sample.png	50000 B	0.000817	0.000835	2.78	11.03
pdf	sample.pdf	200000 B	0.002102	0.002307	2.33	11.15
mp4	large_video.mp4	5000000 B	0.059587	0.055151	2.55	15.11
sql	database_backup.sql	10000000 B	0.097954	0.099284	2.47	18.59

Hash Function Performance Comparison (1 MB Data)

Algorithm	Data Type	Hash Time (s)	Verify Time (s)	CPU (%)	Mem (MB)
MD5	1 MB Random	0.017457	0.017609	1.74	13.63
SHA-1	1 MB Random	0.015659	0.016005	1.99	13.69
SHA-256	1 MB Random	0.029002	0.028726	2.38	13.20
SHA-512	1 MB Random	0.035089	0.034406	2.78	12.60



References

[1] M. K. Mageshwaran and P. S. Ramya, “Data Protection in the Era of Big Data Advanced Encryption Standard VS Data Encryption Standard,” *2024 Second International Conference Computational and Characterization Techniques in Engineering & Sciences (IC3TES),* 2024 .

[2] B. Sankar and S. D, “Advanced Encryption Standard Algorithm in Comparison with FERNET Algorithm for Reducing Encryption Time of H.264 Encoded Video Files,” *2024 IEEE 9th International Conference on Engineering Technologies and Applied Sciences (ICETAS),* 2024.

[3] H. C. Lagunzad, M. V. Gonzaga and C. M. Samonteza, “Development and Implementation of File Security Using Raspberry pi with an Advanced Encryption Standard (AES) Algorithm,” *2023 IEEE 6th International Conference on Pattern Recognition and Artificial Intelligence (PRAI),* 2023.

[4] Y. Qingmin and W. Sinan, “Signature system based on RSA algorithm and its algorithm optimization,” 2021 IEEE Conference on Telecommunications, Optics and Computer Science (TOCS), 2021.

[5] A. A. Asaker, Z. F. Elsharkawy, S. Nassar, N. Ayad, O. Zahran and F. E. A. El-Samie, “A Novel Iris Cryptosystem Using Elliptic Curve Cryptography,” 2021 9th International Japan-Africa Conference on Electronics, Communications, and Computations (JAC-ECC), 2021.

[6] A. Yadav, P. Sharma and Y. Gigras, “A Comparative Study of Elliptic curve and Hyperelliptic Curve Cryptography Methods and an Overview of Their Applications,” 2024 International Conference on Intelligent Systems for Cybersecurity (ISCS), 2024.

[7] M. M. Saqlain and S. Ramachandran, “Highly Secure AES with Key Generation Using SHA-256 Hash Function,” 2025 11th International Conference on Communication and Signal Processing (ICCSP), 2025.

MD5

[8] Wikipedia contributors, "MD5," *Wikipedia, The Free Encyclopedia*, https://en.wikipedia.org/wiki/MD5.